

# Fine-Tuned Transformers versus Prompted LLMs for Software Requirements Classification: An Empirical Study of Generalizability and Cost

**Abstract**—Recent studies report that prompted large language models (LLMs) match or outperform fine-tuned transformers on software requirements classification, and recommend prompting when labeled data is scarce. These results come almost entirely from evaluations within a single dataset, and cost is rarely measured. We compare twelve configurations, four fine-tuned encoder variants and eight prompted LLMs ranging from 4-bit open-weight models on a free GPU to a commercial API, on 1,412 de-duplicated requirements from PROMISE\_exp and SecReq, across three tasks and three levels of distribution shift: in-domain, cross-project and cross-dataset. All 62,406 predictions are scored with one strict rule, and each encoder–LLM comparison is a McNemar test on identical items, corrected over 200 tests. In-domain, the encoders win 46 of 76 tests and lose none. Under cross-dataset transfer the ranking reverses: the prompted models win 48 of 64 tests, and the best in-domain encoder loses 42.8% of its macro-F1. The evidence points to a miscalibrated class prior rather than unfamiliar vocabulary, in both model families. The transferred encoder labels 44.2% of items as security against a true prevalence of 12.9%, close to the 39.9% of its training corpus. The prompted models’ FR/NFR scores track how far their predicted class share is from the truth (Spearman  $\rho = 0.95$ ), and rewording the instruction changes macro-F1 by up to 0.485. The encoders remain 10.7 to 31.0 times cheaper and recover their training cost within 46 to 515 requirements. Fine-tuning suits a stable, labeled domain; prompting is safer when the data distribution is expected to change.

**Keywords**—*requirements classification, large language models, fine-tuning, generalization, distribution shift, empirical software engineering, cost-efficiency*

## I. INTRODUCTION

Requirements classification assigns software requirements to categories such as functional (FR) or non-functional (NFR), or security-relevant or not [1]. It is one of the earliest analysis tasks in requirements engineering (RE), and it becomes laborious and error-prone when a project has hundreds or thousands of requirements. Correct classification supports early risk identification and architectural decisions, while errors propagate downstream as rework [2]. Automated classification was introduced to make the task faster, more consistent and scalable. The techniques have moved from indicator terms [2] to supervised learning [3], fine-tuned transformers [4] and, since 2024, prompted LLMs [5], [6].

Recent LLM studies report that, on the corpora they use, prompting matches or exceeds a fine-tuned transformer [7], [8], and that generative models are competitive without task-specific training [9]; earlier, zero-shot embedding methods had shown that labeled data is not strictly required [10]. These studies conclude that prompting can replace supervised fine-tuning when labeled data is scarce [7]. This conclusion rests on

two assumptions that have not been tested. First, almost all published results are measured within a single dataset. Where generalization is reported, it refers to transfer across tasks inside that dataset, not to unseen projects or to a corpus annotated by another team [7]. Second, running cost is rarely reported; only one of the surveyed studies discusses it, and without measurement [11].

In this paper, we test both assumptions in one controlled experiment. Twelve configurations classify the same requirements from two public corpora on three tasks: FR/NFR, security/non-security, and NFR sub-type classification. Four configurations are fine-tuned encoder-only transformers (BERT and RoBERTa, each with and without inverse-frequency class weighting), and eight are prompted LLMs. All configurations are scored on the same items with one label vocabulary and one scoring rule, and the main experimental factor is the distance between the training and test data. We address three research questions. RQ1: Do prompted LLMs match fine-tuned encoders on in-domain requirements? RQ2: How much macro-F1 do the encoders lose on unseen projects and on an independently annotated corpus, and how do the prompted LLMs compare under the same shift? RQ3: What does each approach cost per requirement, and at what volume does fine-tuning pay for itself?

## II. RELATED WORK

Early work on requirements classification used indicator-term rules, which also produced the PROMISE NFR collection [2], followed by supervised classifiers over hand-crafted features [3]. This section focuses on the two approaches that practitioners now choose between: fine-tuned encoder-based transformers and prompted LLMs.

Hey et al. introduced NoRBERT, which fine-tunes BERT on PROMISE and reports F1 scores of up to 94% [4]; on project-specific folds, where the test projects are not seen during training, its scores are about five points lower. Kaur and Kaur identify transformer fine-tuning as the dominant technique for requirements classification [6]. Vasudevan et al. compare fine-tuned BERT and RoBERTa with GPT-4o and find that the model architecture affects the outcome [12]. None of these studies evaluates outside the corpus used for training, and none reports running cost.

The study closest to ours is by Binkhonain and Alfayez [7], who evaluate five LLMs with four prompting strategies on three classification tasks from PROMISE and SecReq against a fine-tuned transformer, and find that few-shot prompting matches or exceeds the transformer. Alhoshan et al. present the first systematic study of open generative models for requirements classification [9]. They find that the prompt formulation is decisive on the binary tasks and that non-generative models still perform better. Tang et al. report an over-prompting effect [8], which our few-shot experiment reproduces and relates to the class balance of the examples. Requirements classification has also been studied across classical and generative techniques [5], in a systems

engineering workflow [11], on app-store reviews [13], and against expert judgment [14].

Table I summarizes what these studies evaluate. Almost all evaluations stay within one dataset; the exception is the app-review study [13], which uses a different kind of text and reports lower precision on several categories. Cost is rarely reported, even by the studies that use open models [8], [9]. Moreover, the same few small, public and aging corpora underlie nearly every result, so pre-training contamination is acknowledged but not addressed [15]. These gaps reinforce each other: in-domain agreement is the main reason given for replacing supervision with prompting, yet it is also what repeated use of the same corpora would produce.

TABLE I

EVALUATION DIMENSIONS COVERED BY RECENT LLM STUDIES. ✓ = YES, ✗ = NO, ~ = PARTLY.

| Study                    | Open LLMs | Comm. LLMs | Cross-project | Cross-dataset | Cost |
|--------------------------|-----------|------------|---------------|---------------|------|
| El-Hajjani et al. [5]    | ✗         | ✓          | ✗             | ✗             | ✗    |
| Binkhonain & Alfayez [7] | ✓         | ✓          | ✗             | ✗             | ✗    |
| Alhoshan et al. [9]      | ✓         | ✗          | ✗             | ✗             | ✗    |
| Tang et al. [8]          | ✓         | ✓          | ✗             | ✗             | ✗    |
| Peer et al. [11]         | ✗         | ✓          | ✗             | ✗             | ~    |
| Vasudevan et al. [12]    | ✗         | ✓          | ✗             | ✗             | ✗    |
| Chaudhary et al. [13]    | ✗         | ✓          | ✗             | ~             | ✗    |

### III. STUDY DESIGN

The study is a controlled experiment in which twelve model configurations classify the same requirements, on the same tasks and frozen splits, and are scored with one strict rule, so every comparison we report is paired on identical items. Fig. 1 shows the workflow. The factor we vary is the distance between training and test data, at three levels: in-domain (training and test items from the same corpus), cross-project (test projects unseen during training), and cross-dataset (training on one corpus and testing on the other).

**Data preparation.** We use two existing datasets. PROMISE\_exp [16] contributes 969 requirements from 47 projects, labeled FR/NFR and, for NFRs, with one of eleven quality sub-types. SecReq [17] contributes 510 sentences from three industrial security specifications, labeled security-relevant or not. We normalized the texts, mapped both label vocabularies onto one, and removed exact duplicates: 67 of the

1,479 rows were duplicates, which leaves 1,412 requirements. All splits were drawn once with a fixed seed and stored, so fold membership is the same in every run. Two properties of the corpora shape the design. First, security prevalence is 12.9% in PROMISE\_exp and 39.9% in SecReq, a three-fold difference. Second, the security label is defined differently. In PROMISE\_exp, security is one of eleven NFR sub-types, whereas SecReq marks any sentence that raises a security concern and assigns no FR/NFR category. Because binary security classification is the only target the two corpora share, all cross-dataset evaluations use the security task.

**Tasks and label sets.** FR/NFR uses the original PROMISE\_exp labels for all 968 items. The security label is native in SecReq and derived from the sub-type label in PROMISE\_exp, which gives the same task on two independently annotated corpora. Sub-type classification uses the eleven-class taxonomy of Cleland-Huang et al. [2], whose categories correspond to the ISO/IEC 25010 product-quality characteristics [18], and covers the 524 PROMISE\_exp items with a sub-type label. The three tasks are run independently rather than as a pipeline. Every requirement, including the 444 functional ones in PROMISE\_exp, goes directly to the security classifier, and the sub-type classifier receives the items whose gold label is NFR, not those predicted as NFR by the FR/NFR classifier. We did not chain the tasks for two reasons. First, FR/NFR errors would propagate to the later tasks and could not be separated from their own errors. Second, SecReq has no FR/NFR labels, so the first stage of such a pipeline could not be trained or evaluated on it.

**Levels of distribution shift.** The in-domain level uses stratified 5-fold cross-validation, the setting reported by nearly every study in Table I. The cross-project level holds out whole projects, so no requirement from a test project appears in training. A leave-one-project-out variant of FR/NFR is also run, but it is used only in the cost analysis. The cross-dataset level trains on the security task of one corpus and tests on the other, in both directions. The two corpora differ in their annotators and in how they define the security class, so this level combines a shift in definition with a shift in prevalence. Encoders are retrained at each level, so their scores change between levels. Prompted models have no training set, so Table III gives them a single value across the level columns. Because the prompted models stay fixed, a drop in encoder scores can be attributed to the shift and not to harder test items.

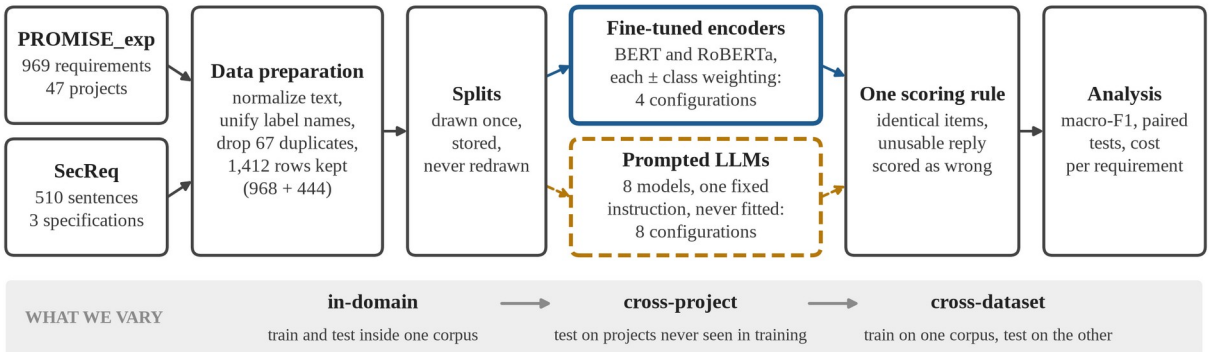


Fig. 1. Experimental pipeline. Both model families classify the same items under one scoring rule; the levels differ in the distance between training and test data.

**Models and sampling.** Table II lists the twelve configurations. Because Vasudevan et al. found that architecture matters [12], we first describe how the two families differ. A fine-tuned encoder is a bidirectional

encoder-only transformer [19], [20]. It pools a representation of the whole requirement into a linear classification head trained on the labeled folds, so both its decision boundary and its estimate of class prevalence are learned from data. A

prompted LLM is a decoder-only autoregressive model that is not adapted to the target dataset. It receives the label set in the prompt and classifies using its pre-trained knowledge and the instructions. We fine-tune BERT-base-uncased [19] and RoBERTa-base [20], following the configurations reported in those papers, with a classification head (AdamW, learning rate  $2 \times 10^{-5}$ , batch size 16, maximum length 128, 3 epochs on the binary tasks and 10 on sub-types). Each encoder is trained with inverse-frequency class weighting; the unweighted version serves as an ablation and is run only on the security task, at the in-domain and cross-dataset levels. Six instruction-tuned open models are run locally: Gemma-2-2B, SmoLM3-3B, Qwen2.5-3B, Phi-4-mini, Qwen2.5-7B and Llama-3.1-8B, with 2.6 to 8.0 B parameters. All six use 4-bit NF4 weight quantization with double quantization and float16 compute [21] on a single free-tier NVIDIA T4, which is what allows an 8 B model to fit in the T4’s 16 GB of memory. The other two models run unquantized at their provider’s default precision: Llama-3.3-70B through the Groq free API tier and Gemini 3.1 Flash-Lite through the Google GenAI free API tier. Decoding is the same for all eight models: greedy, with temperature 0 and at most twelve new tokens. A ninth prompted model was also run but failed the reporting criteria defined below; its predictions are included in the prediction and cell counts given there, but it appears in no table. Encoders classify every requirement once per fold. The API models classify a stratified sample instead, with at least ten items per class: up to 300 requirements per binary task on the hosted tier (286 to 294 were returned) and 60 per task on the commercial tier. The six local models classify every item not held out as a few-shot example, 960 of the 968 in PROMISE\_exp and 436 of the 444 in SecReq. In every significance test, the encoder is scored on exactly the items that its prompted counterpart answered, so all tests are paired.

TABLE II

THE TWELVE CONFIGURATIONS COMPARED, IN TEN ROWS: EACH ENCODER ROW COVERS A WEIGHTED (W) AND AN UNWEIGHTED (U) VARIANT. B = PARAMETERS IN BILLIONS.

| Configuration         | Checkpoint / endpoint   | B    | How it was run  |
|-----------------------|-------------------------|------|-----------------|
| BERT (w / u)          | bert-base-uncased       | 0.11 | fine-tuned      |
| RoBERTa (w / u)       | roberta-base            | 0.13 | fine-tuned      |
| Gemma-2-2B            | gemma-2-2b-it           | 2.61 | free GPU, 4-bit |
| SmoLM3-3B             | SmoLM3-3B               | 3.08 | free GPU, 4-bit |
| Qwen2.5-3B            | Qwen2.5-3B-Instruct     | 3.09 | free GPU, 4-bit |
| Phi-4-mini            | Phi-4-mini-instruct     | 3.84 | free GPU, 4-bit |
| Qwen2.5-7B            | Qwen2.5-7B-Instruct     | 7.62 | free GPU, 4-bit |
| Llama-3.1-8B          | Llama-3.1-8B-Instruct   | 8.03 | free GPU, 4-bit |
| Llama-3.3-70B         | llama-3.3-70b-versatile | 70.6 | hosted API      |
| Gemini 3.1 Flash-Lite | gemini-3.1-flash-lite   | n/a  | commercial API  |

**Prompting.** Each prompted LLM receives one fixed instruction in a single user message. The instruction names the task, lists the permitted labels, and asks for a one-word answer. Fig. 2 shows the FR/NFR prompt; the other tasks use the same template with their own labels.

You are classifying a software requirement.  
Decide whether it is a FUNCTIONAL requirement (FR) - something the system must DO - or a NON-FUNCTIONAL requirement (NFR) - a quality, constraint or property such as performance, security or usability.  
Answer with exactly one word: "FR" or "NFR".

[inserted only in the few-shot conditions, k = 2:]

Examples:

Requirement:

""The system shall have a MDI form that allows for the viewing of the graph and the data table.""

Answer: FR

Requirement:

""The system shall refresh the display every 60 seconds.""

Answer: NFR

Now classify this one:

Requirement:

""The product shall be able to be used by 90% of novice users on the Internet.""

Answer:

Fig. 2. The base FR/NFR prompt as sent to the model. The block marked in brackets appears only in the few-shot conditions (the bracketed line itself is not sent); the requirements shown are real PROMISE\_exp items.

The instruction is kept fixed across levels as an experimental control. We do not claim that it is optimal, and re-optimizing it for each level would confound the prompt with the shift. Its format follows the zero-shot instructions of the studies we compare against [7], [8], and it was never tuned on test items. To estimate how much a better prompt could gain, two models (Gemma-2-2B and Qwen2.5-7B) were re-run with three wordings of the instruction (base, terse and verbose) and the same answer format. Choosing the best wording in hindsight improves macro-F1 by at most 0.020, because the base wording was already at or near the best in almost every case, and such a choice would in practice require labeled data from the target corpus. A second experiment repeats the runs in a few-shot setting, with class-balanced labeled examples inserted before the test requirement: k = 2, 4 or 8 on the binary tasks and k = 1 or 2 on the sub-type task. The examples come from a pool set aside before the evaluation sets were formed, so no example is also a test item.

**Scoring and statistics.** An encoder always returns a permitted label, whereas a prompted model returns free text, so one fixed parsing rule is applied to all outputs. If a reply starts with a label, that label is taken; otherwise, if it names exactly one label, that label is taken. A reply that names two labels or none is counted as an error. This rule is stricter than the protocols of [7], [9] and can only penalize the prompted models, but it reflects how the output would be used in practice. All scores are macro-F1, the unweighted mean of per-class F1 on a 0–1 scale, which weights rare and common classes equally. We also report three percentages. The relative loss is the drop in macro-F1 divided by the in-domain score, so that models with different in-domain scores remain comparable. The predicted class share is the fraction of test items that a model assigns to a class, and the true prevalence is the fraction of items that actually belong to it.

Each encoder–LLM pair is compared with an exact McNemar test on the items where the two models disagree [22]. The test compares item-level correctness rather than macro-F1, but on our data the two agree: all 134 significant results match the macro-F1 ordering. Each test compares one pair of models on one task, corpus and level, and the number of tests follows from the rows of Table III. In-domain, there are 32 tests on PROMISE\_exp security (four encoder rows  $\times$  eight prompted models), 16 each on SecReq security and FR/NFR (two encoder rows each), and 12 on sub-types (two encoder rows  $\times$  the six prompted models reported there), 76 in total. The cross-project level has two encoder rows per column, because the unweighted controls were run only in-domain and cross-dataset (hence the dashes in Table III), which gives 60 tests. The cross-dataset level has four encoder rows on each corpus, which gives 64 tests. All 200 tests are corrected as one family. We compute both Holm (family-wise error) [23] and Benjamini–Hochberg (false-discovery rate) [24] adjustments and report the latter at  $\alpha = 0.05$ , which suits this number of comparisons; the Holm results are stored as well. 95% confidence intervals come from 2,000 stratified bootstrap resamples, and Section IV reports them for the two scores that the cross-dataset result depends on. A result for one model, task, corpus and level (a cell) is reported only if every class has at least five test items, the cell has at least 40 items, and at least 50% of the model’s replies could be parsed. Of the 63 candidate cells, 5 fail these criteria and are excluded (not imputed), leaving 58. Table III reports the 56 cells that fall

within its three levels. They are computed from 62,406 stored predictions, 21,136 from the encoders and 41,270 from the prompted models.

**Cost accounting.** Every stored prediction includes token counts, wall-clock latency and, for encoders, training time. The encoders and the six open-weight LLMs ran on a free Kaggle T4, so there is no invoice. We therefore price their machine time at the published Amazon EC2 on-demand rate for a g4dn.xlarge instance (one NVIDIA T4, us-east-1), \$0.5260 per hour as of 6 August 2026, with requirements classified one at a time. The two API models are priced per million input and output tokens using the public rates of August 2026 (\$0.25/\$1.50 for Gemini 3.1 Flash-Lite and \$0.59/\$0.79 for Llama-3.3-70B), applied to the stored token counts. All prompted runs were made on 7 August 2026, one day after the rates were checked. The provider has since retired the hosted Llama-3.3-70B endpoint, which does not affect the stored predictions or token counts. Cost rises with model size (Spearman  $\rho = 0.72$  over the ten locally run configurations), so in Section IV we treat the ratios between tiers as more stable than the absolute prices. Separating the one-off training cost  $C_0$  from the per-requirement cost  $c_{\text{enc}}$  gives the break-even volume  $N^*$ , in requirements, against a prompted alternative that costs  $c_{\text{alt}}$  per requirement:

$$N^* = C_0 / (c_{\text{alt}} - c_{\text{enc}}) \quad (1)$$

#### IV. RESULTS AND DISCUSSION

TABLE III

MACRO-F1 PER CONFIGURATION, TASK AND LEVEL. EACH COLUMN GROUP NAMES THE CORPUS A MODEL IS SCORED ON, SO AN XD VALUE IS A MODEL TRAINED ON THE OTHER CORPUS. ID = IN-DOMAIN, XP = CROSS-PROJECT, XD = CROSS-DATASET; BEST IN-DOMAIN SCORE PER TASK IN BOLD, “—” = NOT REPORTED.

| Configuration           | Security — PROMISE_exp |       |       | Security — SecReq |       |       | FR/NFR — PROMISE_exp |       | NFR sub-types |       |
|-------------------------|------------------------|-------|-------|-------------------|-------|-------|----------------------|-------|---------------|-------|
|                         | ID                     | XP    | XD    | ID                | XP    | XD    | ID                   | XP    | ID            | XP    |
| BERT <sub>w</sub>       | <b>0.923</b>           | 0.865 | 0.528 | 0.836             | 0.588 | 0.690 | <b>0.908</b>         | 0.843 | 0.657         | 0.579 |
| BERT <sub>a</sub>       | 0.907                  | —     | 0.534 | —                 | —     | 0.702 | —                    | —     | —             | —     |
| RoBERTa <sub>w</sub>    | 0.902                  | 0.876 | 0.698 | 0.824             | 0.668 | 0.696 | 0.896                | 0.853 | <b>0.724</b>  | 0.688 |
| RoBERTa <sub>a</sub>    | 0.903                  | —     | 0.707 | —                 | —     | 0.678 | —                    | —     | —             | —     |
| Gemini 3.1 Flash-Lite   | 0.829                  | —     | —     | 0.713             | —     | —     | 0.867                | —     | —             | —     |
| Llama-3.3-70B           | 0.891                  | —     | —     | 0.814             | —     | —     | 0.852                | —     | —             | —     |
| Qwen2.5-7B              | 0.884                  | —     | —     | <b>0.846</b>      | —     | —     | 0.656                | —     | 0.576         | —     |
| Llama-3.1-8B            | 0.872                  | —     | —     | 0.845             | —     | —     | 0.682                | —     | 0.629         | —     |
| Gemma-2-2B              | 0.900                  | —     | —     | 0.821             | —     | —     | 0.663                | —     | 0.527         | —     |
| Phi-4-mini              | 0.798                  | —     | —     | 0.635             | —     | —     | 0.727                | —     | 0.599         | —     |
| SmolLM3-3B              | 0.841                  | —     | —     | 0.717             | —     | —     | 0.562                | —     | 0.482         | —     |
| Qwen2.5-3B              | 0.707                  | —     | —     | 0.741             | —     | —     | 0.554                | —     | 0.543         | —     |
| majority-class baseline | 0.466                  | —     | —     | 0.376             | —     | —     | 0.351                | —     | 0.035         | —     |

**In-domain comparison.** Table III reports macro-F1 for every configuration, task and level. Encoders achieve the highest in-domain macro-F1 in three of the four task columns: class-weighted BERT on FR/NFR (0.908) and PROMISE\_exp security (0.923), and class-weighted RoBERTa on sub-types (0.724). The exception is SecReq security, where the 4-bit Qwen2.5-7B reaches 0.846 against 0.836. Larger prompted models do not score consistently higher; for example, Gemma-2-2B (2.6 B) scores 0.900 on PROMISE\_exp security, against 0.891 for the hosted 70 B Llama-3.3 model. The encoder margin is largest on the eleven-class task, where the best encoder leads the best prompted model (Llama-3.1-8B, 0.629) by 0.095, which is consistent with supervised training helping most on rare classes. Fig. 3 summarizes the 200 paired McNemar tests by level. In the 76 in-domain tests the encoders win 46 and lose none, and the remaining 30 are not significant.

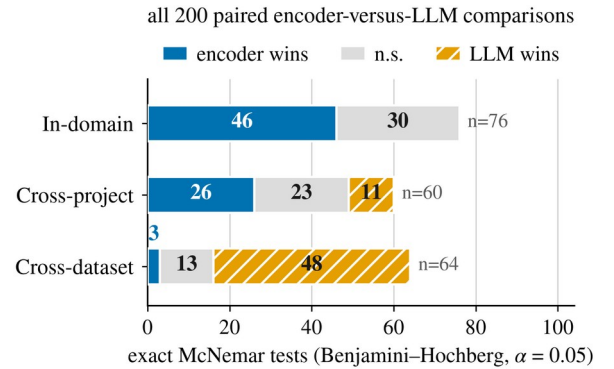


Fig. 3. Outcomes of the 200 paired McNemar tests, grouped by level of shift.

**Generalization.** On PROMISE\_exp security, the eight prompted models range from 0.707 to 0.900 (Table III). Held-out projects cost the encoders little. On PROMISE\_exp, no

encoder loses more than 11.9% of its in-domain macro-F1 (the worst case is BERT on sub-types, from 0.657 to 0.579). The largest cross-project loss in the study is BERT on SecReq, from 0.836 to 0.588 (a relative loss of 29.6%); SecReq has only three projects, so each cross-project fold holds out about a third of the corpus. At this level the encoders are ahead of, or not significantly different from, the prompted models in 49 of the 60 tests, a moderate degradation of the kind Hey et al. reported for unseen projects [4]. The second corpus reverses the result. Class-weighted BERT trained on PROMISE\_exp scores 0.923 (95% CI 0.898–0.946) on PROMISE\_exp security. When it is trained on SecReq and scored on the same items, it drops to 0.528 (0.501–0.555), a relative loss of 42.8%, and RoBERTa drops from 0.902 to 0.698, a loss of 22.7%. The prompted models have no training corpus, so their scores change only with the difficulty of the test corpus. Over the same configurations and scoring, the in-domain count of 46 encoder wins, 30 non-significant results and 0 losses becomes 3 wins, 13 non-significant results and 48 losses. This reversal does not depend on the small API samples: the prompted model wins all 24 cross-dataset tests run on the 960-item cells, while the 60-item commercial cells contribute only 8 of the 64 tests.

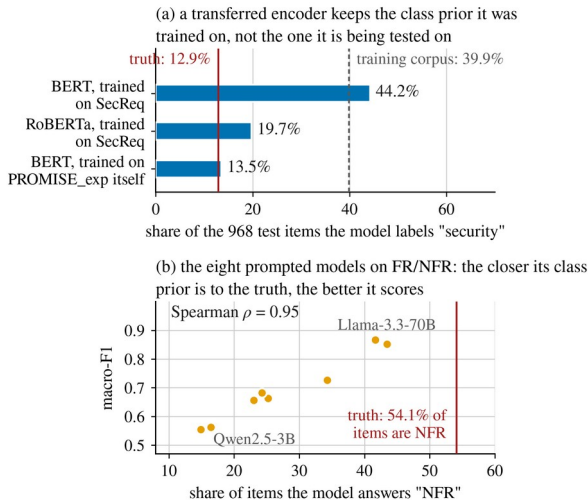


Fig. 4. Share of items labeled security by each encoder (a), and macro-F1 against the share of items answered NFR for each prompted LLM (b).

**Why the encoders lose under transfer.** A drop in macro-F1 does not reveal its cause, and the two likely causes call for different remedies. Unfamiliar vocabulary in the target corpus would require broader training data, whereas a wrong estimate of class prevalence can be corrected without target labels [25]. The predicted class shares in Fig. 4(a) help separate the two. Its three bars correspond to three cells of the PROMISE\_exp security column. Class-weighted BERT trained on PROMISE\_exp labels 13.5% of the items as security, close to the true prevalence of 12.9%, and scores 0.923. Trained on SecReq, where the prevalence is 39.9%, it labels 44.2% as security and scores 0.528. Class-weighted RoBERTa trained on SecReq labels 19.7% and scores 0.698. Across these three cells, the closer the predicted share is to the true prevalence, the higher the score. The last two models share the training and test corpora, and hence the same definitional mismatch, so this mismatch cannot explain the gap between them; they differ in how much of the source prior they carry over. The per-class results point the same way. For BERT, security precision falls from 0.85 in-domain to 0.21 after transfer, while recall falls much less, from 0.89 to 0.72. This pattern is consistent with

over-prediction of the security class rather than a failure to recognize security requirements.

Class weighting does not fix the transfer, and its effect is in line with the prior-shift explanation. Inverse-frequency weighting increases the tendency to predict the minority class of the training corpus. When a model is trained on SecReq (39.9% security) and tested on PROMISE\_exp (12.9%), weighting therefore pushes the predicted share further from the target, and the unweighted controls score slightly higher (0.534 vs. 0.528 for BERT and 0.707 vs. 0.698 for RoBERTa). In the reverse direction weighting should help, and it does for RoBERTa (0.696 vs. 0.678) but not for BERT (0.690 vs. 0.702). All four differences are at most 0.018 macro-F1, more than twenty times smaller than the loss that the transfer itself causes for BERT. Reweighting the training data cannot provide the class prior of the test data.

The prompted models show the same pattern, although they have no training corpus. On FR/NFR, 54.1% of the items are NFR, and every prompted model answers NFR less often than that. Macro-F1 rises almost monotonically as a model's NFR share approaches the true prevalence (Spearman  $\rho = 0.95$ , Fig. 4(b)), so the weak FR/NFR scores of the small open models are consistent with a strong bias toward answering FR.

For the prompted models, the instruction wording changes the predicted share directly, so here the link can be tested by intervention; for the encoders it is inferred from the observed pattern. On SecReq security, the three wordings change Gemma-2-2B's macro-F1 by up to 0.485, from 0.343 to 0.828 on a 300-item sample. Under the terse wording, which asks a yes/no question, the model labeled 96.7% of the items as security, while the prevalence in that sample is 39.7%. Prompt sensitivity is well documented [26]; our results show that it acts through the same class-prior error seen in the encoders. Few-shot examples help only where they add information about class balance: they improve the eleven-class task and slightly degrade the binary tasks, which reproduces the over-prompting effect reported by Tang et al. [8]. In short, both families need an estimate of class prevalence for the data being classified, which no single requirement provides, and both take it from a source other than the target data.

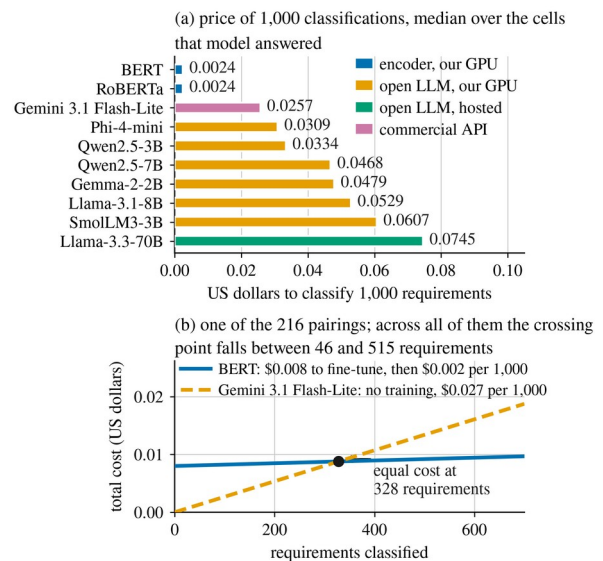


Fig. 5. Cost of 1,000 classifications per configuration (a), and the volume at which one encoder becomes cheaper than one prompted LLM (b).

**Cost.** Classifying 1,000 requirements costs \$0.0024 of GPU time with an encoder and between \$0.0257 and \$0.0745 with a prompted model (median per model), 10.7 to 31.0 times more (Fig. 5(a)). Fine-tuning one deployable model costs a median of \$0.0083. Over the 216 encoder–LLM pairs (the 200 tested above and 16 from the leave-one-project-out variant), the break-even volume  $N^*$  from (1) lies between 46 and 515 requirements, with a median of 177.5. Fig. 5(b) shows one of these pairs, class-weighted BERT against Gemini 3.1 Flash-Lite on FR/NFR, where the two cost lines cross at 328 requirements. Above 515 requirements, the largest break-even volume we observed, the encoder is cheaper in every pair. At 10,000 requirements, the median total cost is \$0.03 for the encoder, including its one-off training, and \$0.26 for the commercial API. Within each task and corpus, 7 of the reported cells are Pareto-optimal, meaning that no other cell has both a higher macro-F1 and a lower cost. Six of them are encoders; the only prompted one is the 4-bit Qwen2.5-7B on SecReq security, the one column where a prompted model already leads in-domain. These figures cover computation only. The cost of labeling a few hundred requirements, which only the encoder approach needs, is not included, and it favors prompting.

**Implications for evaluation.** The main finding is that the ranking of the two families reverses between two evaluation settings that the literature treats as interchangeable, even though both families are scored on the same items with the same labels and scoring rule. This has three implications. 1) Reported results should state the evaluation setting together with the metric. 2) The term generalization should refer to held-out projects or an independently annotated corpus rather than to cross-task transfer within one dataset [7]. 3) The mechanism can be checked on any new corpus pair without re-running these models, by comparing a model’s predicted share of the rare class with its true prevalence in the target data. Since the cross-dataset comparison covers one corpus pair, one task (security) and one language, the study shows that the reversal occurs and can be large, but not how often it occurs.

**Practical guidance.** The choice between the two approaches should depend on the data situation more than on model size. When a few hundred labeled requirements are available and the volume is non-trivial, a 110 M-parameter fine-tuned encoder leads in three of the four task columns at an order of magnitude lower cost. When the data are unlabeled or the distribution is likely to shift, a mid-size quantized open model gives up some in-domain macro-F1 but keeps its score when the corpus changes, since it was never fitted to a particular corpus, and it requires no API contract. Model size is not decisive: the hosted 70 B model was the most expensive one we measured and did not score highest. Our results caution against fine-tuning on one corpus and assuming that the model will transfer.

## V. THREATS TO VALIDITY

**Construct validity.** Costs are estimated from prices rather than taken from invoices: GPU costs use a published rental rate and API costs use 2026 list prices.  $N^*$  covers computation only and ignores differences in accuracy between options, so it indicates when fine-tuning becomes cheaper, not when it becomes preferable. The strict parsing rule (Section III) could turn the measure partly into one of output reliability, since only prompted models can produce an unusable reply. On the binary tasks, however, every reported prompted model returned a usable label for all of its items, so the rule affects

only the sub-type task, where at least 94.7% of each model’s replies were usable.

**Internal validity.** The API models answered the stratified samples described in Section III rather than the full data, and the commercial tier only 60 items per cell. Each encoder is scored on exactly the same items, so the comparisons remain paired, but the sampled cells have larger estimation error. Free-tier rate limits caused some requests on the hosted tier to fail. Cells that then fell below the reporting criteria were excluded rather than imputed, and the remaining cells are scored on the items that were returned. Each encoder configuration was fine-tuned with a single seed, so each cross-dataset score comes from one training run and its variance across seeds is not estimated. The prompted models use greedy decoding, so their outputs are essentially deterministic, but the local models ran with 4-bit quantization, which may lower their scores slightly compared with full precision. Duplicates were removed before any split was drawn. All 67 duplicates were within a corpus, and no text occurs in both corpora, so no item can leak from a training corpus into a test corpus.

**Conclusion validity.** The commercial-tier cells are small, and 19 of the 22 comparisons involving that tier are not significant. Correcting 200 tests as one family lowers power further. Low power hides significant differences in both directions: it understates the 46 encoder wins, but it also makes it easier for the encoders to record no significant losses, so this record partly reflects limited power. Neither issue affects the cross-dataset result, which rests on 48 of 64 tests rather than on a single borderline comparison.

**External validity.** Both corpora are small, dated and public, and they define the security class differently (Section III), so the cross-dataset level involves a shift in definition as well as in distribution, and it is tested on only one corpus pair. The public corpora may be part of the prompted models’ pre-training data [15]. This would inflate their scores equally at every level; it cannot explain why the ranking changes between levels, which is driven by the drop in encoder scores. The study covers English text and one NFR taxonomy. The pipeline, stored splits, predictions and analysis scripts will be released upon acceptance, so that every reported value can be recomputed from the raw predictions.

## VI. CONCLUSION AND FUTURE WORK

We compared four fine-tuned encoder configurations with eight prompted LLMs on three requirements classification tasks, scoring both families on the same items with one label vocabulary and one scoring rule while varying the distance between training and test data. In-domain, the encoders lead and, at the available statistical power, are never significantly outperformed. On an independently annotated corpus, the ranking reverses. The encoders remain an order of magnitude cheaper to run and recover their training cost within a few hundred classifications. The results are consistent with a single mechanism in both families: each takes its estimate of class prevalence from a source other than the corpus being classified. A result measured within one dataset can therefore lead to the wrong choice between the two approaches.

We see two directions for future work. **Fine-tuning the LLMs.** The prompted models were used off the shelf, so the comparison is one-sided: the encoders are adapted to the task and the LLMs are not. Alhoshan et al. fine-tune generative models in-domain [9], but transfer remains open. Training

LoRA adapters on the same 4-bit checkpoints [21], retrained at each level like the encoders, would show whether the prompted models transfer better because they are generative or simply because they were never fitted to a source corpus. It would also add a training cost for this family, which is currently priced for inference only.

**Cross-dataset evaluation with PURE.** Since the reversal was observed on one corpus pair, the next step is to annotate a third source for the security task, such as PURE, a public collection of complete requirements documents [27]. This would provide several cross-dataset comparisons instead of one, allow the prevalence gap between training and test corpora to be varied systematically, and add a corpus that is less likely to have been seen during pre-training.

#### ACKNOWLEDGMENT

Generative AI tools were used to assist in drafting and editing the text of this paper. The authors are responsible for the study design, the experiments and all reported results.

#### REFERENCES

- [1] “ISO/IEC/IEEE International Standard — Systems and software engineering — Life cycle processes — Requirements engineering,” in ISO/IEC/IEEE 29148:2018(E), pp. 1–104, 30 Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.
- [2] J. Cleland-Huang et al., “Automated classification of non-functional requirements,” *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, 2007.
- [3] Z. Kurtanović and W. Maalej, “Automatically classifying functional and non-functional requirements using supervised machine learning,” in *Proc. IEEE RE*, 2017, pp. 490–495.
- [4] T. Hey et al., “NoRBERT: Transfer learning for requirements classification,” in *Proc. IEEE RE*, 2020, pp. 169–179.
- [5] A. El-Hajjaji, N. Fafin, and C. Salinesi, “Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT,” *arXiv:2311.11547*, 2024.
- [6] K. Kaur and P. Kaur, “The application of AI techniques in requirements classification: A systematic mapping,” *Artif. Intell. Rev.*, vol. 57, art. 57, 2024, doi: 10.1007/s10462-023-10667-1.
- [7] M. Binkhonain and R. Alfayez, “Are prompts all you need? Evaluating prompt-based large language models (LLMs) for software requirements classification,” *Requirements Eng.*, vol. 30, no. 3, pp. 423–443, 2025, doi: 10.1007/s00766-025-00451-8.
- [8] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, “The few-shot dilemma: Over-prompting large language models,” *arXiv:2509.13196*, 2025.
- [9] W. Alhoshan, A. Ferrari, and L. Zhao, “How effective are generative large language models in performing requirements classification?” *arXiv:2504.16768*, 2025.
- [10] W. Alhoshan, A. Ferrari, and L. Zhao, “Zero-shot learning for requirements classification: An exploratory study,” *Inf. Softw. Technol.*, vol. 159, art. 107202, 2023.
- [11] J. Peer, Y. Mordecai, and Y. Reich, “NLP4ReF: Requirements classification and forecasting — From model-based design to large language models,” in *Proc. IEEE Aerospace Conf.*, 2024, pp. 1–16.
- [12] P. Vasudevan, S. Reddivari, S. Ahuja, and N. Niu, “Requirements classification using fine-tuned transformer models: A comparative study of BERT, RoBERTa, and GPT-4o,” in *Proc. ACM Southeast Conf. (ACMSE)*, 2026, pp. 199–204, doi: 10.1145/3746467.3801509.
- [13] M. Chaudhary, C. Jain, and P. R. Anish, “Exploring zero-shot app review classification with ChatGPT: Challenges and potential,” *arXiv:2505.04759*, 2025.
- [14] O. Levy, I. Dikman, N. Levy, and M. Winokur, “AI-assisted requirements engineering: An empirical evaluation relative to expert judgment,” *arXiv:2604.15222*, 2026.
- [15] O. Sainz et al., “NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark,” in *Findings of EMNLP*, 2023, pp. 10776–10787.
- [16] M. Lima et al., “Software engineering repositories: Expanding the PROMISE database,” in *Proc. SBES*, 2019, pp. 427–436.
- [17] E. Knauss et al., “Supporting requirements engineers in recognising security issues,” in *Proc. REFSQ*, 2011, pp. 4–18.
- [18] ISO/IEC 25010:2011, *Systems and Software Engineering — SQuARE — System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.
- [19] J. Devlin et al., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [20] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv:1907.11692*, 2019.
- [21] T. Dettmers et al., “QLoRA: Efficient finetuning of quantized LLMs,” in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 10088–10115.
- [22] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [23] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
- [24] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
- [25] M. Saerens, P. Latinne, and C. Decaestecker, “Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure,” *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002.
- [26] Z. Zhao et al., “Calibrate before use: Improving few-shot performance of language models,” in *Proc. ICML*, 2021, pp. 12697–12706.
- [27] A. Ferrari, G. O. Spagnolo, and S. Gnesi, “PURE: A dataset of public requirements documents,” in *Proc. IEEE RE*, 2017, pp. 502–505.